

10149: Yahtzee

- ★★☆☆☆
- 題組：Problem Set Archive with Online Judge
- 題號：10149: Yahtzee
- 解題者：蔡閔仰
- 解題日期：2014年3月26日
- 題意：給你13組隨機骰子(5個一組)，玩Yahtzee算出可能的最高分。規則如下頁所示：

13格種類裡面分為上半部的 6 個計分格和下半部的 7 個計分格：

上半部的六個計分格有：

Ones Twos Threes Fours Fives Six

他們的計分規則很簡單 根據你骰出的結果，只取那個數字的骰子計分，比如說，你骰出的結果是 一 一 三 三 五。

如果記在 **Ones** 的話，就是2分（有兩個一的骰子）

Threes 6 （有兩個三的骰子）

Fives 5 （只有一個五的骰子）

下半部的七個計分格有：

7.**Chance**（機會）：記所有骰子點數和

8.**Three of a kind**（三條）：至少三個相同，記所有點數和

9.**Four of a kind**（四條;鐵支）：至少四個相同，記所有點數和

10. **Five of a kind** (五條)：全相同，計五十分

11. **Small Straight** (小順子)：四個點數成順，計25分
(ex : 1,2,3,4)

12. **Large Straight** (大順子)：五個點數成順，計35分
(ex : 1,2,3,4,5)

13. **Full House** (葫蘆)：三個相同點數和兩個相同點數，
計40分 (ex : 1,1,1,2,2)

- 後六種計分項中，若不滿足條件，則計0分。
- 前六種計分項之得分種和 ≥ 63 ，則總分在加35分。
- 每個計分項只能使用一次
- 有13組骰子

任務：計算遊戲可得之最高總分

■ 題意範例：1 2 3 4 5(*13)→1 2 3 4 5 0 15 0 0 0 25 35 0 0 90

組 輸入

1 : 11111 -> C10

8 : 61266 -> C7

2 : 66666 -> C6

9 : 14555 -> C5

3 : 66611 -> C8

10: 55556 -> C9

4 : 11122 -> C13

11: 44456 -> C4

5 : 11123 -> C1

12: 31363 -> C3

6 : 12345 -> C12

13: 22246 -> C2

7 : 12346 -> C11

輸出：

C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	加分	總分
----	----	----	----	----	----	----	----	----	-----	-----	-----	-----	----	----

3	6	9	12	15	30	21	20	26	50	25	35	40	35	327
---	---	---	----	----	----	----	----	----	----	----	----	----	----	-----

(輸出以一個空白為分隔)(C1~C6和為75，且和 >= 63)

■ 解法：

C1 C2 C3 ...(計分項)

(組) -----

R1 | a1 a2 a3 ...

R2 | b1 b2 b3 ...

R3 | c1 c2 c3 ...

$C_i(1 \sim 13)$ 代表第*i*種計分方式， $R_i(1 \sim 13)$ 代表第*i*組骰子。

窮舉法：共有 $13! = 62,2702,0800$ 種取法，多次重覆計算，這種方法雖然可行，但是會超出時間。

Dynamic Programming (動態規劃)：用DP解，將13種解法分別對應成2進位形式，有 $2^{13} = 8192$ 種取法，此舉我們稱作**掩碼(bitmask)**，

例如： $(00000000000001)_2$ ($C_{13} \sim C_1$)，代表我們的第一組骰子選了第一種取法。(1的多寡代表做到第幾組)

(1所在的位元代表用過了哪些計分項)

而我們可以將那2進位的掩碼轉換成十進位的n，也就是sum[1₂][s]就假設紀錄我們使用選取第一種取法的策略且此此時前六種取法的總分是s(此項目的是為了判斷最後是否有Bonus)，所以sum[11111111111111₂][s]就是我們最後要的總分。

依據s會有兩種可能出現最高分的情形，一種是沒有加分($s < 63$)，另一種是加分($s \geq 63$)。若 ($s \geq 63$) $\Rightarrow$ 把總和放在 sum[?][63]，而小於就放在index裡。

最後我們可以找到最高總分，但題目還要求我們連每個計分項的分數都要印出，因此，需要一個紀錄步驟的陣列。

EX: 設陣列trace[a][b][2]，a、b存 sum 的兩個 index，而最後一個索引[0]存當下這個[a][b]是前面哪個策略過來的，[1]則存前一個狀態的前六項得分。

■ 解法範例：

//遍歷所有可能的計分組合方式，並計算該最大值

```
for(int m = 0;m<combinations(2^13);m++)
```

```
    for(int c=0;c<categories(13);c++)
```

```
        if(!(m&(1<<c))) { /*確保第(c+1)種策略沒用過*/
```

```
            bits_num=countbits(m); /*算m2有幾個1*/
```

```
            s=score[bits_num][c]; /* 代表第(?+1)組骰子
```

```
                的第(c+1)種策略得分 */
```

```
            t=m | (1<< c); /* t記錄策略跟新後的掩碼 */
```

```
            a = (c<6) ? s : 0; /*判斷是否為前六種策略*/
```

```
            for(int u = 0; u < bonus(64); u++) {
```

```
                /* 以下就是遍歷 */
```

```
if(sum[m][u]>-1){  
    d=((u+a)<(bonus-1)) ? (u+a) : (bonus-1);  
    if(sum[t][d]<(sum[m][u]+s)){  
        trace[t][d][0] = c; /* 此兩行是紀錄追溯資訊 */  
        trace[t][d][1] = u;  
        sum[t][d] = sum[m][u] + s; /* 更新總分 */  
    }  
}
```


■ 討論：

1. 使用掩碼(bitmask)並用DP解。
2. 位元處理：

運算子：

>> 右移 、 << 左移 、 & AND 、 | OR

& AND 範例：

特色：判斷 n 變數 的二進位 有幾個 1

```
int c = 0;
```

```
for( int i = 0; i < 32 ; ++i)  /* int 變數有32個位元 */
```

```
    if( n & (1 << i))
```

```
        c++;
```

| OR 範例：

特色：把位元強制標成 1

ex:

$n | (1 \ll 4)$ /* 把變數n第五位元標成 1